Monte-Carlo Value-at-Risk (VaR) Engine

Jolan Lebehot

2026-10-28

Contents

1 Theoretical Foundations **2**

1.1 Log Returns vs. Simple Returns . . . 2

1.2 Time additivity . . . 2

1.3 Price Positivity Issue . . . 2

2 From Historical Data to Stochastic Modeling **4**

2.1 Drift of the assets . . . 4

2.2 Volatility of the asset . . . 4

2.3 Geometric Brownian Movement . . . 5

3 Data Extraction and Empirical Implementation **8**

3.1 Basic implementations . . . 8

3.2 Monte Carlo simulation . . . 10

3.3 Value at risk engine . . . 12

4 Conclusion **15**

1 Theoretical Foundations

1.1 Log Returns vs. Simple Returns

In quantitative finance and risk simulation, log returns are usually preferred over simple returns but I wondered why. That's why I analyzed these 2 types of return to know which one is more practical for our using.

First of all let's introduce these types of return, if we consider that the price of the asset at time t is P_t, the simple return between time $t-1$ and time t is written :

$$R_t = \frac{P_t - P_{t-1}}{P_{t-1}}$$

and the log return between time $t-1$ and time t is written :

$$r_t = \ln\left(\frac{P_t}{P_{t-1}}\right)$$

1.2 Time additivity

Let us start with log returns, if we want the log return between time t and time $t+n$ with $n \in \mathbb{N}$,

$$r_{total} = \ln\left(\frac{P_{t+n}}{P_t}\right) = \sum_{k=1}^{n} \ln\left(\frac{P_{t+k}}{P_{t+k-1}}\right) = \sum_{n=1}^{n} r_{t+k}$$

so the formula is quite simple.

About simple returns, if we want the simple return between time t and time $t+n$ with $n \in \mathbb{N}$,

$$\frac{P_t}{P_{t-1}} = 1 + R_t \quad \Longrightarrow \quad \frac{P_{t+n}}{P_t} = (1+R_{t+1})...(1+R_{t+n})$$

and so

$$R_{total} = \prod_{k=1}^{n}(1+R_{t+k}) - 1$$

that is analytically more complex.

1.3 Price Positivity Issue

Furthermore, with simple returns, if we want the asset's price at time $t+n$, it is given by the formula :

$$P_{t+n} = P_t(1+R)$$

but returns will be generated by probability distribution supported on $\mathbb{R}$. It creates an issue if there are values such that $R < -1$ because it leads to a negative asset price P_{t+n} that is financially inconsistent.

In the other side, with log returns, the asset's price at time $t + n$, we have :

$$P_{t+n} = P_t \exp(R) > 0, \quad \forall \in \mathbb{R}$$

So this financial inconsistency of single returns disapears with log returns.

2 From Historical Data to Stochastic Modeling

While historical analysis with log returns provide empirical insights into past assets behaviors, risk management require more projections. To simulate portfolio's paths, we need to model the millions of possibilities that the assets could have in the futur.

Let us now transition from historical prices P_t to a stochastic process denoted by S_t to model future price dynamics.

2.1 Drift of the assets

If we imagine that a financial asset grows in a perfectly predictable way, with an average rate μ, its time price would be :

$$\frac{dS_t}{dt} = \mu S_t \quad \Leftrightarrow \quad dS_i = \mu S_t dt \quad \Leftrightarrow \quad \frac{dS_t}{S_t} = \mu dt$$

So, integrating between time t and time $t + n$,

$$\ln\left(\frac{S_{t+n}}{S_t}\right) = r_{total} = n\mu$$

It leads to the definition of the drift μ :

$$\mu = \frac{1}{n} r_{total} = \frac{1}{n} \sum_{n=1}^{n} r_{t+k}$$

as the average of log returns by the time additivity of log returns.

2.2 Volatility of the asset

Nevertheless, in real life, markets are pretty unpredictable. To model this randomness, we are adding a random shock. Obviously, these variations depend on the price S_t of the asset at time t, because it should definitively have more absolute variations on a larger asset than on a smaller one. (for instance, there would more variations in a 500$ asset than on a 10$ one)

It also depends on the past volatility σ that we can define as the standard variation of log returns.

So these shocks can be described by the term :

$$\sigma S_t dW_t$$

where dW_t represent pure market noise, daily chaos, or the unexpected that we are calling the Brownian movement.

Let us examine how dW_t, the daily chaos, can be modeled. First of all, in a long period, this unexpected has not a privileged direction, that is why

$$\mathbb{E}(dW_t) = 0$$

Then, the more the time interval is large, the more the unexpected one should be high. Why is the variance proportional to time? Intuitively, market uncertainty accumulates over time: predicting an asset's price tomorrow carries less dispersion than predicting it a year from now. Formally, the variance of the Brownian increment scales linearly with the time step dt, meaning

$$\mathbb{V}(dW_t) = dt$$

Furthermore, these shocks are caused by the countless actions taken by buyers, sellers and banks through the world. We can suppose that because of the uncertainty surrounding these actions, the increments are independent and identically distributed (i.i.d.) with no privileged direction.

Therefore, over a time interval $\Delta t = n\,dt$, by invoking the Central Limit Theorem for the sum of these n small increments:

$$Z_t = \frac{\frac{1}{n}\sum_{i=0}^{n-1} dW_{t+idt} - \mathbb{E}(dW_t)}{\sqrt{\mathbb{V}(dW_t)}/\sqrt{n}} = \frac{\sqrt{n}}{\sqrt{dt}}\frac{1}{n}\sum_{i=0}^{n-1} dW_{t+idt} = \frac{1}{\sqrt{ndt}}\sum_{i=0}^{n-1} dW_{t+idt}$$

such that $Z_t \sim \mathcal{N}(0,1)$ as n grows large.

This implies that the aggregated finite-horizon increment follows a normal distribution:

$$\sum_{i=0}^{n-1} dW_{t+idt} \sim \mathcal{N}(0, ndt = \Delta t)$$

This gives us the theoretical justification for the distribution of Brownian movements over any time horizon Δt.

2.3 Geometric Brownian Movement

Adding the 2 terms gives us the equation :

$$dS_t = \mu S_t dt + \sigma S_t dW_t$$

which is the Geometric Brownian Movement (GBM).

This differential equation mixed the price of the asset S_t and the Brownian Movement dW_t and we do not know how to solve a differential equation like that. Îto's lemma

That is why we are going to use a lemma which will help us:

Îto's lemma For $f(X)$ a $\mathcal{C}^2(\mathbb{R}_+, \mathbb{R})$, we have the formula

$$d(f(X)) = \frac{\partial f}{\partial X} dX + \frac{1}{2} \frac{\partial^2 f}{\partial X^2} (dX)^2$$

If we take the function $f(S_t) = \ln(S_t)$ (we are calling the function ln to have the log return between time t and time $t + \Delta t$ as we will see later) and we apply Îto's lemma to this $\mathcal{C}^2(\mathbb{R}_+, \mathbb{R})$ function, it leads to

$$d(\ln(S_t)) = \frac{1}{S_t} dS_T - \frac{1}{2} \frac{1}{S_t^2} (dS_t)^2$$

because

$$\frac{\partial \ln}{\partial S_t}(S_t) = \frac{1}{S_t} \quad \text{et} \quad \frac{\partial^2 \ln}{\partial S_t^2}(S_t) = -\frac{1}{S_t^2}$$

So if we replace $dS_t = \mu S_t dt + \sigma S_t dW_t$ in this equation,

$$d(\ln(S_t)) = \mu dt + \sigma dW_t - \frac{1}{2} \frac{1}{S_t^2} (dS_t)^2$$

Nevertheless,

$$(dS_t)^2 = (\mu S_t dt + \sigma S_t dW_t)^2 = \mu^2 S_t^2 (dt)^2 + 2\mu\sigma S_t^2 dW_t dt + \sigma^2 S_t^2 (dW_t)^2$$

with $\mathbb{V}(dW_t) = \mathbb{E}(dW_t^2) - (\mathbb{E}(dW_t))^2 = \mathbb{E}(dW_t^2) - 0 = dt$ and we suppose that the variations are insignificant because $\sigma = \sqrt{\mathbb{V}(dW_t)} = \sqrt{dt} = o(dt)$, so we can assume that $dW_t^2 \simeq dt$ and $dW_t \simeq \sqrt{dt}$.

It leads to the fact that we can ignore two terms when $dt \to 0$ in the equation of $(dS_t)^2$, the first one is the one in $(dt)^2$ because $(dt)^2 = o(dt)$ and the second one is the one in $dW_t dt$ because $dW_t dt \simeq (dt)^{3/2} = o(dt)$.

And so

$$(dS_t)^2 = \sigma^2 S_t^2 (dW_t)^2 \simeq \sigma^2 S_t^2 dt$$

It leads to

$$d(ln(S_t)) = \mu dt + \sigma dW_t - \frac{1}{2}\sigma^2 dt = \left(\mu - \frac{1}{2}\sigma\right) dt + \sigma dW_t$$

Let us integrate between time t and time $t + \Delta t$

$$\int_t^{t+\Delta t} d(\ln(S_t)) = \int_t^{t+\Delta t} \left(\mu - \frac{1}{2}\sigma\right) dt + \int_t^{t+\Delta t} \sigma dW_t$$

$$\Leftrightarrow \quad \ln\left(\frac{S_{t+\Delta t}}{S_t}\right) = \left(\mu - \frac{1}{2}\sigma\right) \Delta t + \sigma(W_{t+\Delta t} - W_t)$$

As established in the previous section, the integral of the infinitesimal increments over the finite horizon Δt represents the accumulated Brownian increment. Building directly on our result regarding the sum of independent increments, this finite-horizon term follows:

$$W_{t+\Delta t} - W_t \sim \mathcal{N}(0, \Delta t)$$

Let us denote this random shock by $Z_{\Delta t} = W_{t+\Delta t} - W_t \sim \mathcal{N}(0, 1)$.

So it leads to the log return

$$r = \ln\left(\frac{S_{t+\Delta t}}{S_t}\right) = \left(\mu - \frac{1}{2}\sigma\right) \Delta t + \sigma Z_{\Delta t}$$

with $Z_{\Delta t} \sim \mathcal{N}(0, \Delta t)$.

And we can easily extract from the formula

$$S_{t+\Delta t} = S_t \exp(r)$$

3 Data Extraction and Empirical Implementation

3.1 Basic implementations

To get all these data, we need to import the yfinance module and other modules in our Python code.

```
import yfinance as yf
import numpy as np
import math
import numpy.random as rd
import matplotlib.pyplot as plt
```

Then we need to define a portfolio of stocks for which we are going to extract the data we need to do the study. So we need to choose the firm that we are going to study and give the function the period of time and the interval of each measure.

```
tickers = 'AAPL'
# Let us take the example of one Apple share

def extraire(portfolio, period, interval):
    data = yf.download(portfolio, period=period, interval=interval, auto_adjust=False)['Adj Close']
    # We use Adjusted Close to avoid biasing the volatility calculation.

    return data.squeeze()

data = extraire(tickers, '2y', '1d')
```

```
## [**********************100%**********************]  1 of 1 completed
```

```
print(data)
```

```
## Date
## 2024-10-28    231.463760
## 2024-10-29    231.731522
## 2024-10-30    228.191132
## 2024-10-31    224.035889
## 2024-11-01    221.060791
##                    ...
## 2026-10-22    348.220001
## 2026-10-23    347.730011
## 2026-10-26    343.910004
## 2026-10-27    347.100006
```

```
## 2026-10-28    345.271606
## Name: AAPL, Length: 502, dtype: float64
```

To begin, as we studied before, we need to construct a function that calculates log returns.

```
def log_return(data):
    # We know that the log return is ln(P_t / P_{t-1})
    # .shift(1) move the table down by one row to get P_{t-1} in front of P_t
    log_returns = np.log(data / data.shift(1))

    # .dropna() removes the first row from the returns column as there is no previous value
    log_returns = log_returns.dropna()

    return log_returns

print(log_return(data))
```

```
## Date
## 2024-10-29    0.001156
## 2024-10-30   -0.015396
## 2024-10-31   -0.018377
## 2024-11-01   -0.013369
## 2024-11-04   -0.004046
##                  ...
## 2026-10-22    0.012222
## 2026-10-23   -0.001408
## 2026-10-26   -0.011046
## 2026-10-27    0.009233
## 2026-10-28   -0.005282
## Name: AAPL, Length: 501, dtype: float64
```

Let us define the volatility on a period of time by the formula :

$$\sigma = \sqrt{\mathbb{V}(r)}$$

and construct the function of it in Python.

```
def volatility(data):
  return np.std(log_return(data))

print(volatility(data))
```

```
## 0.017965867828617737
```

Then, we are going to introduce the drift of the asset,

```
def drift(data):
  return np.mean(log_return(data))

print(drift(data))
```

```
## 0.0007982196681433342
```

3.2 Monte Carlo simulation

All these basic implementations let us simulate all the future log returns of the asset over the desired time horizon and number of time steps.

```
n = len(data)
dt = 1



def r(data):
  t = []
  v = volatility(data)
  d = drift(data)
  for i in range(n):
    t.append((d- (1/2)*v**2)*dt + v*rd.normal(0,np.sqrt(dt)))
  return t

print(r(data)[:5])
```

```
## [np.float64(5.062876783465246e-05), np.float64(0.006661082322081671), np.float64(-0.0292472876783705
```

Then we can easily know the simulations of the asset's price

```
def p(data):
  t = []
  t.append(np.float64(data.iloc[-1]))
  sim_r = r(data)
  for i in range(n):
    previous = t[-1]
    t.append(previous*np.exp(sim_r[i]))
  return t
```

```
print(p(data)[:5])
```

```
## [np.float64(345.2716064453125), np.float64(350.9024288337638), np.float64(363.76960663610697), np.fl
```

Now, we simulate n times the trajectory of the portfolio's prices

```
ng = 100

simulated_prices = []

for i in range(ng):
    simulated_prices.append(p(data))

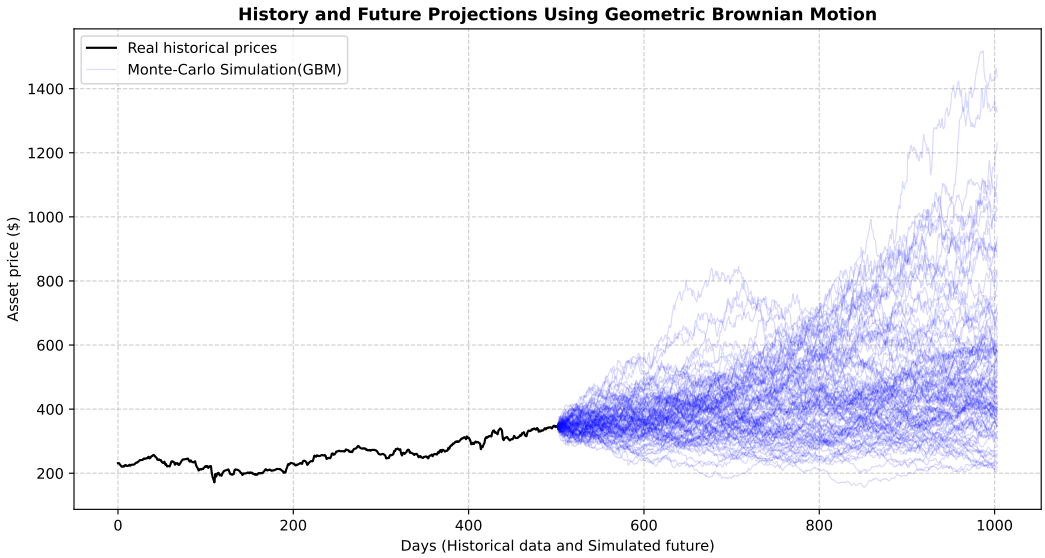

plt.figure(figsize=(12, 6))


hist = data.values
x_hist = range(len(hist))
plt.plot(x_hist, hist, color='black', linewidth=1.5, label='Real historical prices')


last_inst_idx = len(hist) - 1
x_sim = range(last_inst_idx, last_inst_idx + len(simulated_prices[0]))

for i in range(ng):
    # To have a unique legend
    label = 'Monte-Carlo Simulation(GBM)' if i == 0 else ""
    plt.plot(x_sim, simulated_prices[i], color='blue', alpha=0.15, linewidth=0.8, label=label)


plt.title("History and Future Projections Using Geometric Brownian Motion", fontsize=12, fontweight='bo
plt.xlabel("Days (Historical data and Simulated future)", fontsize=10)
plt.ylabel("Asset price ($)", fontsize=10)
plt.grid(True, linestyle='--', alpha=0.6)
plt.legend(loc='upper left')
plt.show()
```



3.3 Value at risk engine

First of all, we need to introduce the profit & loss indicator,

```
def pnl(data):
  initial_price = float(data.iloc[-1])
  final_price = [s[-1] for s in simulated_prices]
  return np.array(final_price) - initial_price

print(pnl(data)[:5])
```

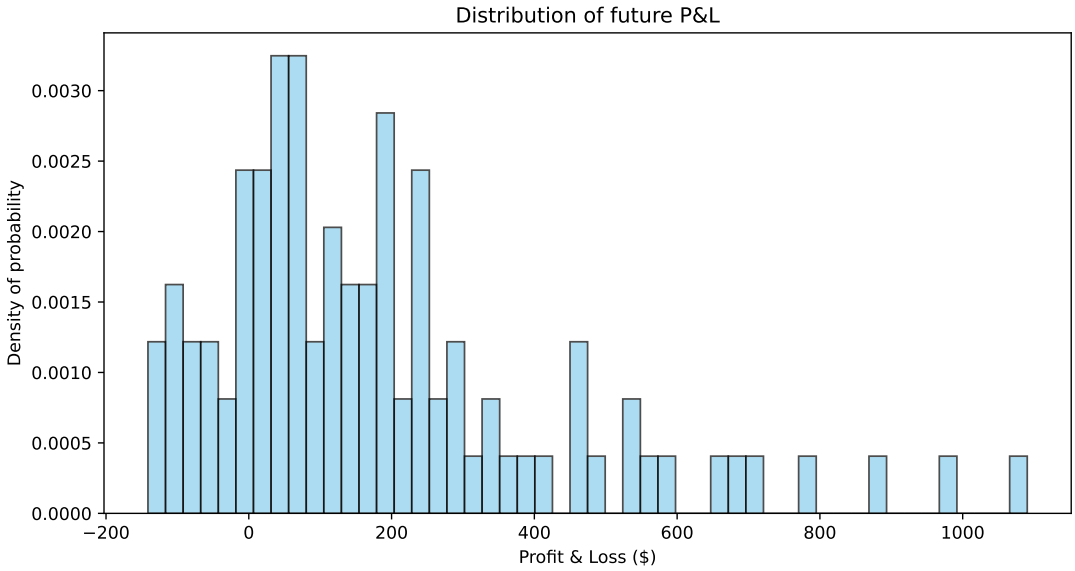
```
## [886.37681036  59.44388667 140.80399242 231.4770096  252.79379325]
```

```
pnl_values = pnl(data)



plt.figure(figsize=(10, 5))
plt.hist(pnl_values, bins=50, color='skyblue', edgecolor='black', alpha=0.7, density=True, label='Distri

plt.title("Distribution of future P&L")
plt.xlabel('Profit & Loss ($)')
plt.ylabel('Density of probability')
plt.show()
```



And we finally introduce the Value at Risk.

```
# Calculating the VaR at 99% and at 95% (= quantile at 99% and at 95%)
var_99 = np.percentile(pnl_values, 1)
var_95 = np.percentile(pnl_values, 5)

print("Value-at-Risk (99%) : ", var_99)
```

```
## Value-at-Risk (99%) :  -132.31205074605847
```

```
print("Value-at-Risk (95%) : ", var_95)
```

```
## Value-at-Risk (95%) :  -111.57817797807061
```

Thus, with a 99% confidence level, the maximum expected loss on a single Apple share over the simulation horizon will not exceed the Value-at-Risk at 99%.

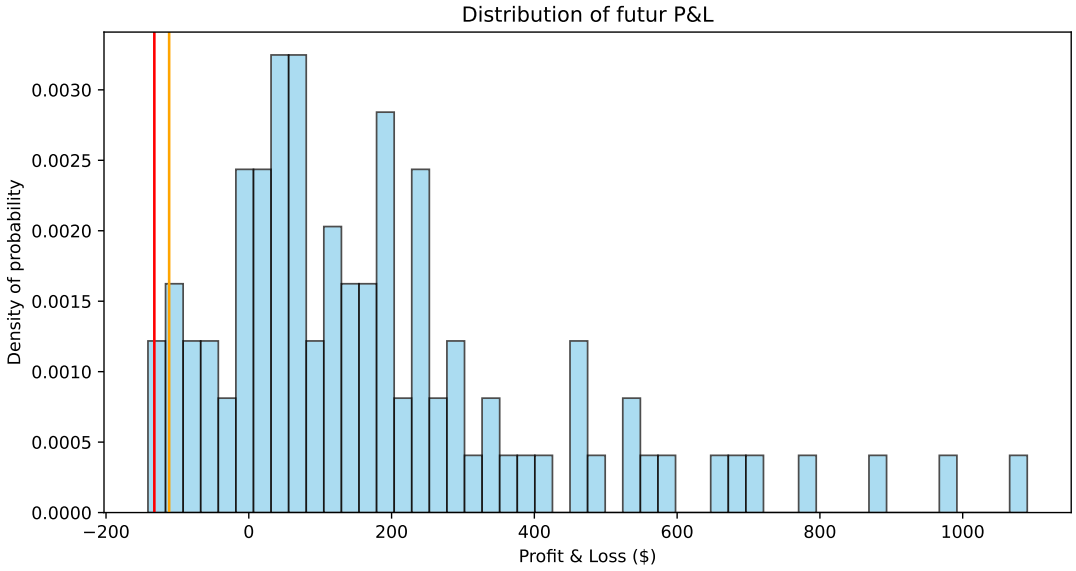
And we can show the VaR on the previous histogram.

```
plt.figure(figsize=(10, 5))
plt.hist(pnl_values, bins=50, color='skyblue', edgecolor='black', alpha=0.7, density=True, label='Distri

plt.axvline(var_99, color='red')
plt.axvline(var_95, color='orange')

plt.title("Distribution of futur P&L")
plt.xlabel('Profit & Loss ($)')
```

```
plt.ylabel('Density of probability')
plt.show()
```



It is clear that the density lies almost entirely to the right of the vertical asymptotes representing the 99% and 95% per cent VaRs.

4 Conclusion

Through this study, we discovered why we work with log returns in finance and we learned how to use it. Then we analyzed historical data to introduce the two notions that are the drift and the volatility that we used to modeling the Geometric Brownian Movement.

To solve this equation, we needed to have the Îto's lemma to model potential futur returns which is leaded by normal distributions. Then we model all the possibilities to predict futur asset's prices.

Finally, we calculated the quantile at 99% and 95% to know the maximum expected loss for a single share over the simulation horizon.